

Workshop 03 – Besprechung der Lösung

Musterlösung: `loesung.py`.

Überblick

Das Skript ist in Funktionen gegliedert, die den Aufgabenschritten entsprechen: `load_data` → `normalize_data` → `prepare_data` → `split_data` → `build_model` → `train_model` / `make_predictions` → `Plots.main()` verdrahtet sie und wertet das `--predict`-Flag aus.

1. Einlesen und normalisieren

```
data = pd.read_csv(file_path)
numeric_data = data.select_dtypes(include=[np.number])
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(numeric_data)
```

`select_dtypes` wirft die Datumsspalte `t` heraus – übrig bleiben 7 numerische Features. `MinMaxScaler` skaliert **jede Spalte unabhängig** auf 0–1 (Volumen und Preise haben völlig unterschiedliche Größenordnungen). Der Scaler wird zurückgegeben, falls man Vorhersagen später zurücktransformieren möchte.

2./3. Sequenzen bilden und aufteilen

```
for i in range(len(data) - input_timesteps - output_timesteps + 1):
    samples.append(data[i : i + input_timesteps])
    labels.append(data[i + input_timesteps : i + input_timesteps + output_timesteps])
```

Ein gleitendes Fenster erzeugt `samples` mit Form (N, 64, 7) und `labels` mit (N, 32, 7). Da eine Dense-Ausgabeschicht einen flachen Vektor liefert, werden die Labels zu (N, 32 * 7) umgeformt:

```
labels = labels.reshape(labels.shape[0], output_timesteps * samples.shape[2])
```

`train_test_split(test_size=0.2, random_state=42)` teilt zufällig – für den Workshop ausreichend. **Hinweis:** Bei Zeitreihen wäre ein chronologischer Split (Training = Vergangenheit, Test = Zukunft) korrekter, weil sich sonst überlappende Fenster in beiden Mengen finden.

4. Modell

```
model = Sequential()
model.add(LSTM(100, activation="relu", return_sequences=True, input_shape=(64, num_features)))
model.add(Dropout(0.2))
model.add(LSTM(100, activation="relu"))
model.add(Dropout(0.2))
model.add(Dense(output_timesteps * num_features))
model.compile(optimizer="adam", loss="mse")
```

- Zwei gestapelte LSTM-Schichten; die erste gibt mit `return_sequences=True` die ganze Sequenz an die zweite weiter, die zweite nur den letzten Zustand.
- `Dropout(0.2)` gegen Overfitting.
- Die Ausgabeschicht hat $32 * 7 = 224$ Neuronen, **ohne** Aktivierung (Regression), Verlust `mse`.

5. Training

```
early_stopping = EarlyStopping(monitor="val_loss", patience=10)
model_checkpoint = ModelCheckpoint("best_model.h5", save_best_only=True, monitor="val_loss", mode="min")
tensorboard = TensorBoard(log_dir="logs")
history = model.fit(X_train, y_train, epochs=50, batch_size=64, validation_split=0.2,
                    callbacks=[early_stopping, model_checkpoint, tensorboard])
```

`validation_split=0.2` zweigt 20 % der Trainingsdaten als Validierung ab. `ModelCheckpoint` sichert das Modell mit dem niedrigsten `val_loss`; `EarlyStopping` beendet das Training, wenn 10 Epochen ohne Verbesserung vergehen. Der `TensorBoard`-Callback (Bonus) schreibt nach `logs/`.

`plot_training_history` zeichnet `loss` und `val_loss`.

6. Vorhersagen

```
model = load_model("best_model.h5")
y_pred = make_predictions(model, X_test)
y_test = y_test.reshape(y_test.shape[0], output_timesteps, num_features)
y_pred = y_pred.reshape(y_pred.shape[0], output_timesteps, num_features)
plot_predictions(y_test, y_pred)
```

Die flachen Vektoren werden auf (N, 32, 7) zurückgeformt; `plot_predictions` zeichnet die geflatteten Werte gegeneinander. Aussagekräftiger wird der Plot, wenn man nur eine Spalte (z. B. `close`, Index 3) und ein einzelnes Sample betrachtet – das ist eine gute Erweiterung für den Bonus.

Ergebnis

Der Verlust fällt schnell auf sehr kleine Werte (alle Features liegen auf 0–1). Die Vorhersage folgt dem Trend, glättet aber kurzfristige Ausschläge – typisch für LSTMs auf Finanzdaten, die überwiegend Rauschen enthalten.